fn new_continuation_rule

Michael Shoemate (@Shoeboxam)

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `fn new_continuation_rule`.

1 Hoare Triple

Precondition

Compiler-verified

Types consistent with pseudocode.

Caller-verified

None

Pseudocode

```
def new_continuation_rule(d_in: MI_Distance, d_out: MO_Distance) -> Wrapper:

    def wrapper(queryable: Queryable) -> Queryable:
        state = queryable
        del queryable  # (moved by rust ownership)

        def transition(query: Query[Any]) -> Answer[Any]:  #

            if state is None:
                raise "filter is exhausted"
            else:
                queryable = state

            # Retain an invocation error until the privacy loss is checked.
            error = None
            try:
                answer = queryable.eval_query(query)
            except Exception as caught_error:
                error = caught_error

            match queryable.eval_poly(
                OdometerQuery.PrivacyLoss(d_in.clone())
            ):  #
                case OdometerAnswer.PrivacyLoss(pending_d_out):
                    pass
                case _:
                    raise "expected privacy loss"

            if pending_d_out.total_gt(d_out):  #
```

```
30                  state.take()  #
31                  raise "filter is now exhausted"
32
33              if error is not None:
34                  raise error  #
35              return answer  #
36
37          return Queryable.new_raw(transition)
38
39      return Wrapper.new(wrapper)  #
```

Postcondition

Theorem 1.1. Returns a function that wraps a queryable. The wrapped queryable refuses to release any query that would cause the privacy loss to exceed `d_out`.

Proof. Line 39 returns a function that wraps a queryable.

The new queryable, whose transition function is defined on line 7, runs the routine on line 23 for every query that changes the privacy loss. This routine queries the original queryable for what the new privacy loss is, after executing the query. If the new privacy loss exceeds `d_out` on line 29, the query is rejected, and the inner queryable is freed from memory on line 30. The invocation outcome is retained until this check has completed, including when the outcome is an error from fallible postprocessing after an internal private release. In particular, an over-budget invocation error is withheld just like a successful answer.

Otherwise, the answer is returned on line 35, or the retained invocation error is raised on line 34. □